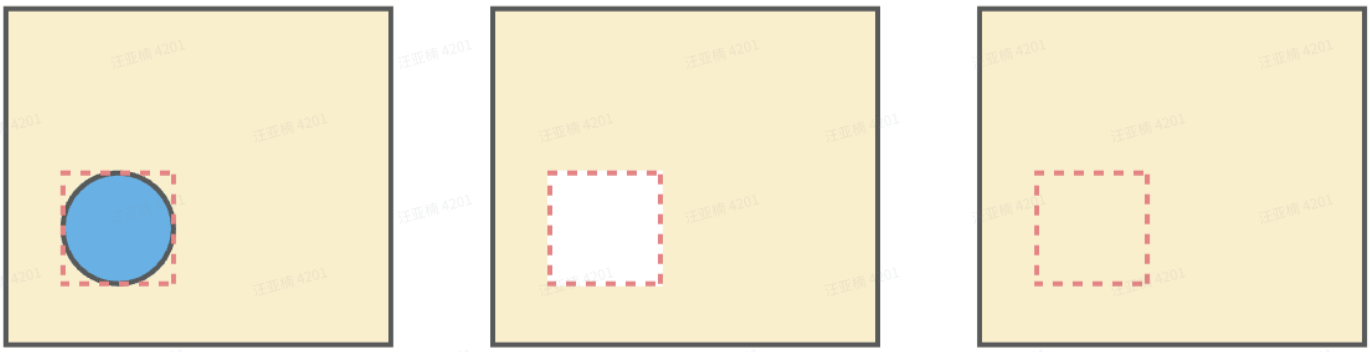
脏区重绘

> 从名字上看，就是让画布只重绘“脏掉的”区域，单从这点出发，你有实现思路了吗？



标记脏区　　清除脏区　　重绘脏区

① 标记脏区域：当画布上的某个区域发生改变时，这个区域就被标记为"脏区"。

② 清除脏区域：在重绘之前，先清除脏区域的当前内容。

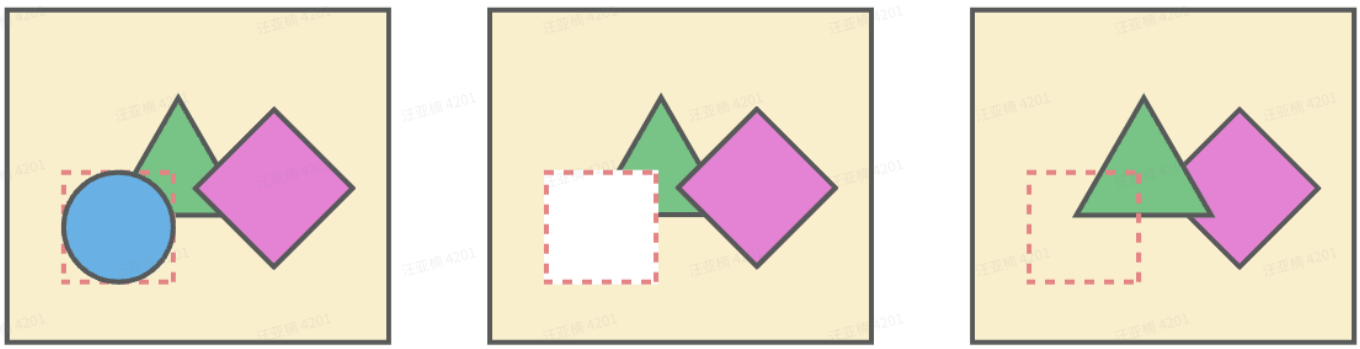
③ 重绘脏区域：使用新的或过滤后的数据重绘脏区域。

从上面的步骤看，“脏区重绘”操作可以减少我们重绘的内容，提高渲染性能，所以是常见的一种画布优化方法。

没那么简单

如果你觉得“脏区重绘”是一个这么容易掌握的方法那我就必须得敲醒你了。

咱们再看一种新的情况：



标记脏区　　清除脏区　　重绘脏区

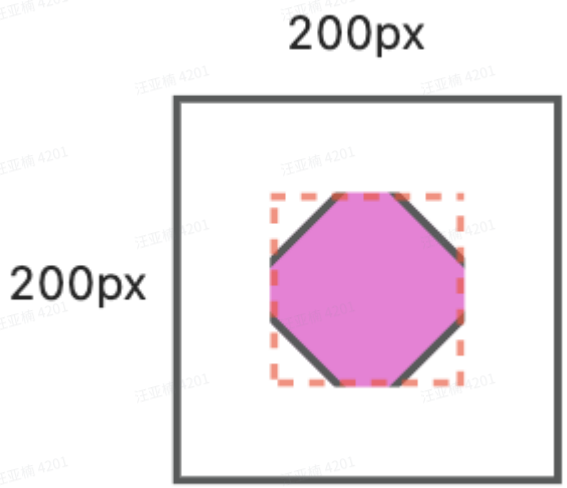
如果按照上面的步骤处理我们会发现最后的效果是错误的，因为我们只重绘了脏区范围内出现过的绿色三角形，使得绿色三角形绘制在紫色方块之上，这改变了原图效果。

这么看来我们是不能过滤元素的，必须所有元素依次重新绘制进画布才能得到原来的效果，但如果是这样，那就达不到“脏区重绘”的目的了。

`clip()` 强力辅助

`CanvasRenderingContext2D.clip()` 可以为画布设置剪切区域，设置后后续画布的绘制都只在剪切区域中有效。

下面的例子就设置了一个 `100 * 100` 的剪切区域（红色虚线框）：




```
const canvas = document.getElementById('canvas');
canvas.width = 200;
canvas.width = 200;
const ctx = canvas.getContext('2d');

ctx.beginPath();
ctx.rect(50, 50, 100, 100);
ctx.clip();
```

> 画布绘制时，绘图引擎会自动剪裁超出部分，因为这可以减少不必要的计算和绘制，从而提高绘制性能。

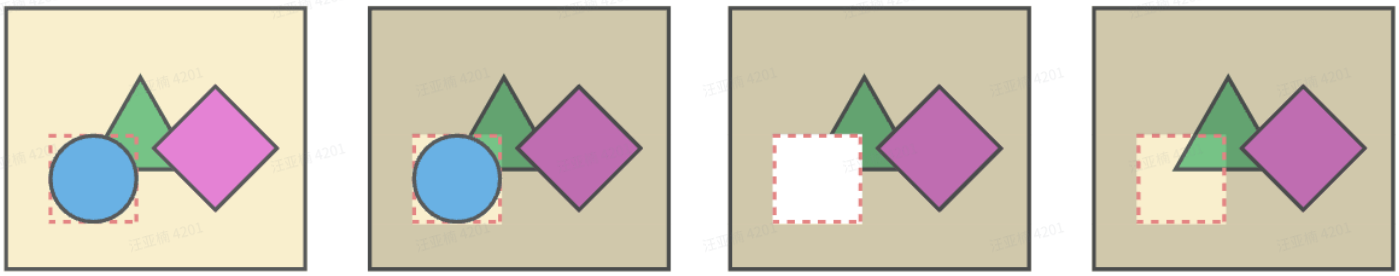
这个是我们在前面的文章里提到的，而这里的 `clip()` 方法设置的剪切区域也属于绘图引擎认定的剪切区域，所以当我们绘制的元素超出这个裁切区域时也能享受到绘图引擎提供的优化服务。结合这一点，我们就能实现一个加强版的“脏区重绘”了。

① 标记脏区域。

② 通过 `clip()` 将脏区域设置成剪切区域

③ 清除脏区域。

④ 重绘脏区域，无需手动过滤超出范围的元素，因为绘图引擎自带裁剪和优化。



标记脏区　设置剪切区域　清除脏区　重绘脏区

`clip()` 性能隐患

虽然 `clip()` 可以实现更好的“脏区重绘”，提高重绘时的渲染性能，但是它本身却带有性能隐患。 `clip()` 创建的剪切区域并不像普通的路径一样可以通过 `beginPath()` 去清除。

```
context.beginPath();
context.rect(50, 50, 200, 200);
context.clip();

context.beginPath();
context.rect(100, 100, 50, 50);
context.clip();
```

像👆的代码，两次 `clip()` 的调用就会生成两个剪切区域，并不是调用 `beginPath()` 后就将前一个剪切区域清除了。

但是如果你将其用 `save()` 、 `restore()` 包裹，即可在使用后重置剪切区域，restore()方法会撤销之前的clip()调用，恢复到保存状态时的裁剪区域。

```
context.save();
context.beginPath();
context.rect(50, 50, 200, 200);
context.clip();
context.restore();
```